SDEV 2401

Rapid Backend App Development

Files and Media - 2

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- Why Upload Non-Image Files?
- Django Settings for File Uploads
- File Fields in Models
- Uploading Files with a Form
- Processing Uploaded Files
- Downloading Files
- Summary

WE ARE ESSENTIAL TO ALBERTA
NAIT

Why Upload Non-Image Files?

- Many apps need to handle files like CSVs, PDFs, DOCX, or ZIPs etc.
- Example: Students upload essays as PDF, or teams share project ZIPs.
- File uploads aren’t just for images—think about assignments, contracts, or code archives.
- As well this is a great way to import data in bulk into your application. For example uploading a CSV file with multiple records to create many objects at once.

Django Settings for File Uploads

You'll need the same setup for images to handle media files in general. This is the same as what we did in the last example.

settings.py

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'user_files'
```

- Store all user-uploaded files in a dedicated folder.

- `MEDIA_URL` is the web path; `MEDIA_ROOT` is the server path.
- Keeps user files separate from app code and static assets.

WE ARE ESSENTIAL TO ALBERTA | NAIT

File Fields in Models: Report & Attachment

`FileField` is used for uploading any file type, not just images. They are perfect for documents, spreadsheets, and anything else you might need to upload.

models.py

```
class Report(models.Model):
    name = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)

class Attachment(models.Model):
    report = models.ForeignKey(Report, on_delete=models.CASCADE)
    file = models.FileField(upload_to='attachments/')
    uploaded_at = models.DateTimeField(auto_now_add=True)
```

- `FileField` lets you upload any file type.
- `upload_to` organizes files in subfolders (you can see this locally in `user_files/attachments/`, we use a slightly different folder in the example we'll do together).
- Each attachment is linked to a report.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Uploading Files with a Form

Uploading a file is similar to uploading an image. You create a form with a `FileField`, handle the upload in the view, and save the file.

forms.py

```
class AttachmentForm(forms.ModelForm):
    class Meta:
        model = Attachment
        fields = ['file']

    # you can add validation here to check file types, size
    # we'll do this in the example.
```

views.py

```
def upload_attachment(request, report_id):
    if request.method == 'POST':
        form = AttachmentForm(request.POST, request.FILES)
        if form.is_valid():
            attachment = form.save(commit=False)
            attachment.report_id = report_id
            attachment.save()
            # Redirect or show success
    else:
        form = AttachmentForm()
    return render(request, 'upload.html', {'form': form})
```

- Always use `request.FILES` for file uploads.
- The form will render a file input for the user.
- Saving the form stores the file in the correct location.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Processing Uploaded Files

Processing uploaded files is common, e.g., reading CSVs to import data, let's take a look at how to do it.

views.py

```
import csv

def process_csv(request):
    if request.method == 'POST' and request.FILES.get('csv_file'):
        csv_file = request.FILES['csv_file']
        decoded = csv_file.read().decode('utf-8').splitlines()
        reader = csv.DictReader(decoded)
        for row in reader:
            # Do something with each row
            # You can create model instances here!
            pass
        # Redirect or show success
    return render(request, 'process_csv.html')
```

- Use Python's built-in `csv` module to read CSV files.
- The `'utf-8'` is the default encoding for text files.

Downloading Files

Just like images, you can provide download links for any uploaded files in your templates the same way.

Template Example

```
{% for attachment in attachments %}
  <a href="{{ attachment.file.url }}">Download {{ attachment.file.name }}</a>
{% endfor %}
```

- `.url` gives the path to the uploaded file.
- Users can download files directly from the template.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Summary

- Use `FileField` for uploading any file type.
- Set up `MEDIA_URL` and `MEDIA_ROOT` in settings for user files.
- Handle uploads in forms with `request.FILES`.
- Process files as needed (e.g., reading CSVs).
- Provide download links in templates using the file's `.url`.

Example

Let’s go do an example together